

Loops: while and for - Simple Hinglish Summary

Source: <https://javascript.info/while-for>

Main baat

Loop ka matlab: **ek hi code ko baar baar chalana**. Jaise list ke items ek ek karke dikhana, ya 1 se 10 tak har number ke liye same kaam karna.

(Is chapter me sirf basic loops hain: `while`, `do..while`, `for`. Objects ke liye `for..in` aur arrays ke liye `for..of` alag chapters me aayenge.)

1. `while` loop

javascript

```
while (condition) {  
  // loop body  
}
```

Jab tak `condition` **truthy** hai, body chalti rehti hai.

javascript

```
let i = 0;  
while (i < 3) {    // 0, 1, 2 dikhayega  
  alert( i );  
  i++;  
}
```

- Body ke ek baar chalne ko **iteration** kehte hain. Upar 3 iterations hue.
- Agar `i++` na ho to loop **hamesha chalta rahega** (infinite loop).
- Condition koi bhi expression ho sakti hai, wo boolean me convert hoti hai. `while (i != 0)` ko chhota karke `while (i)` likh sakte ho:

javascript

```
let i = 3;  
while (i) {  
  alert( i );    // 3, 2, 1  
  i--;  
}
```

- Body me **ek hi statement** ho to `{ }` zaruri nahi:
-

javascript

```
let i = 3;
while (i) alert(i--);
```

2. **do..while** loop

Condition **body** ke baad check hoti hai:

javascript

```
do {
    // loop body
} while (condition);
```

Pehle body chalti hai, phir condition check hoti hai.

javascript

```
let i = 0;
do {
    alert( i );
    i++;
} while (i < 3);
```

Kab use karein: jab body **kam se kam ek baar** chalni hi chahiye, chahe condition true ho ya na ho. Warna aam taur par **while** hi use hota hai.

3. **for** loop

Sabse zyada use hone wala loop.

javascript

```
for (begin; condition; step) {
    // loop body
}
```

javascript

```
for (let i = 0; i < 3; i++) {    // 0, 1, 2
    alert(i);
}
```

Part	Example	Kya karta hai
begin	<code>let i = 0</code>	Loop me aate waqt sirf ek baar chalta hai
condition	<code>i < 3</code>	Har iteration se pehle check hota hai. False ho to loop band
body	<code>alert(i)</code>	Condition true rehne tak chalti hai
step	<code>i++</code>	Har iteration me body ke baad chalta hai

Flow: `begin` (ek baar) → condition check → body → step → condition check → body → step → ... jab tak condition false na ho.

Inline variable declaration

`for` ke andar declare kiya variable **sirf loop ke andar** dikhta hai:

```
javascript
for (let i = 0; i < 3; i++) {
  alert(i);
}
alert(i); // error, i loop ke bahar nahi hai
```

Pehle se bana variable bhi use kar sakte ho:

```
javascript
let i = 0;
for (i = 0; i < 3; i++) { ... }
alert(i); // 3, bahar dikhta hai
```

Parts skip karna

`for` ka koi bhi part chhod sakte ho:

```
javascript
```

```

let i = 0;
for (; i < 3; i++) {    // begin skip
    alert( i );
}

for (; i < 3;) {        // step bhi skip (while jaisa ban gaya)
    alert( i++ );
}

for (;;) {              // sab skip = infinite loop
    // hamesha chalta rahega
}

```

Dhyan: dono semicolons(;) zaruri hain, warna syntax error.

4. **break** (loop tod dena)

Loop normally tab band hota hai jab condition false ho. Lekin **break** se **kabhi bhi turant** band kar sakte ho.

```

javascript

let sum = 0;

while (true) {
    let value = +prompt("Enter a number", '');

    if (!value) break;    // khali input ya cancel par loop band

    sum += value;
}

alert( 'Sum: ' + sum );

```

"Infinite loop + break" tab badhiya hai jab condition **beech me** ya kai jagah check karni ho.

5. **continue** (agle iteration par jao)

break ka halka version. Poora loop nahi rokta, sirf **current iteration chhod** kar agla shuru karta hai.

```

javascript

```

```
for (let i = 0; i < 10; i++) {
  if (i % 2 == 0) continue; // even par skip
  alert(i);                // 1, 3, 5, 7, 9
}
```

`continue` se **nesting kam** hoti hai (`if` block ke andar code ghusane ki zarurat nahi padti), jisse code zyada readable rehta hai.

? ke saath `break/continue` nahi chalta

javascript

```
(i > 5) ? alert(i) : continue; // syntax error
```

`break/continue` expressions nahi hain, isliye ternary **?** me nahi use ho sakte. Ye bhi ek wajah hai ki **?** ko `if` ki jagah use nahi karna chahiye.

6. Labels (`break` / `continue` ke liye)

Nested loops me ek saath bahar nikalna ho to label chahiye. Normal `break` sirf **andar wala loop** todta hai.

Label = loop se pehle `naam:`

javascript

```
outer: for (let i = 0; i < 3; i++) {

  for (let j = 0; j < 3; j++) {

    let input = prompt(`Value at coords (${i},${j})`, '');

    if (!input) break outer; // dono loops se bahar
  }
}

alert('Done!');
```

- `break outer` upar `outer` naam ka label dhundhkar us loop se bahar nikal jata hai.
- `continue outer` bhi ho sakta hai: labeled loop ke agle iteration par jata hai.
- Labels se code me **kahin bhi jump nahi kar sakte**. `break` hamesha code block ke andar hona chahiye. 99.9% baar loops me hi use hota hai.

Summary

Loop	Condition kab check hoti hai
<code>while</code>	Har iteration se pehle
<code>do..while</code>	Har iteration ke baad
<code>for (;;)</code>	Har iteration se pehle (extra settings ke saath)

- **Infinite loop** ke liye aam taur par `while(true)` use hota hai, aur `break` se roka jata hai.
- `continue`: current iteration chhodkar agle par jao.
- **Labels**: nested loop se bahar nikalne ka **ekmatra** tarika.

Practice Tasks (Answers)

1. Aakhri value?

```
javascript
let i = 3;
while (i) {
  alert( i-- );
}
```

Jawab: **1**. Sequence 3, 2, 1, phir `i = 0` par loop band.

2. Prefix vs postfix in while:

```
javascript
let i = 0;
while (++i < 5) alert( i );    // 1 se 4 tak
```

```
javascript
let i = 0;
while (i++ < 5) alert( i );    // 1 se 5 tak
```

Postfix me comparison purani value se hota hai, lekin `alert` naya `i` dikhata hai, isliye 5 bhi dikhta hai.

3. `for` me `i++` aur `++i`:

```
javascript
```

```
for (let i = 0; i < 5; i++) alert( i );
for (let i = 0; i < 5; ++i) alert( i );
```

Dono me **0 se 4** tak. Increment ki return value use nahi ho rahi, isliye fark nahi padta.

4. 2 se 10 tak even numbers:

javascript

```
for (let i = 2; i <= 10; i++) {
  if (i % 2 == 0) {
    alert( i );
  }
}
```

5. `for` ko `while` me badlo:

javascript

```
let i = 0;
while (i < 3) {
  alert( `number ${i}!` );
  i++;
}
```

6. Jab tak 100 se bada number na aaye, poochte raho:

javascript

```
let num;

do {
  num = prompt("Enter a number greater than 100?", 0);
} while (num <= 100 && num);
```

Do checks zaruri hain: `num <= 100` aur `&& num`. Cancel dabane par `num = null` hota hai, aur `null <= 100` true hota hai, isliye `&& num` ke bina loop kabhi nahi rukta.

7. 2 se `n` tak prime numbers:

javascript

```
let n = 10;

nextPrime:
for (let i = 2; i <= n; i++) {

  for (let j = 2; j < i; j++) {
    if (i % j == 0) continue nextPrime;    // prime nahi, agla i
  }

  alert( i );    // prime
}
```

Yaad rakhne wali baatein

- Sabse common: `for`. Condition pehle se pata ho to `while`.
- `break` = loop khatam, `continue` = sirf ye iteration khatam.
- Nested loop se bahar nikalna ho to **label** use karo.